

RevBookMyShow

AWS Deployment Report

Full deployment documentation with screenshots

Prepared for project presentation

Repository: Rev-BookMyShow

Author: Rajendra0309

Contents

1	Project Overview	2
1.1	Final Architecture	2
1.2	Architecture Screenshot	2
2	Prerequisites	2
2.1	Tool Verification	3
3	Backend Deployment	4
3.1	Docker Image Build	4
3.2	ECR Repository	4
3.3	Image Push to ECR	5
3.4	ECS Cluster	5
3.5	Task Definition	6
3.6	Application Load Balancer	7
3.7	Target Group	8
3.8	ECS Service	8
3.9	Backend Health Verification	9
4	Frontend Deployment	9
4.1	Frontend API Configuration	9
4.2	Frontend Build	10
4.3	S3 Bucket Creation	10
4.4	Static Website Hosting	10
4.5	Public Access and Bucket Policy	11
4.6	Uploading Dist Files	12
4.7	Frontend Live URL	12
5	CORS and Environment Matching	13
6	Troubleshooting Notes	14
6.1	S3 AccessDenied	14
6.2	Unhealthy Target Group	14
6.3	CORS Failure	14
6.4	Mixed Content Error	15
7	Conclusion	15

1. Project Overview

RevBookMyShow is a movie ticket booking application deployed on AWS using a simple and reliable architecture. The frontend is hosted on Amazon S3 Static Website Hosting, while the backend runs on Amazon ECS Fargate behind an Application Load Balancer. The backend image is stored in Amazon ECR, and MongoDB connection details are injected through ECS task environment variables.

1.1. Final Architecture

- Frontend: Amazon S3 Static Website Hosting
- Backend: Amazon ECS Fargate
- Container Registry: Amazon ECR
- Load Balancer: Application Load Balancer
- Database: MongoDB Atlas or external MongoDB URI
- Health Check: `/health`
- API Path: `/api`

1.2. Architecture Screenshot

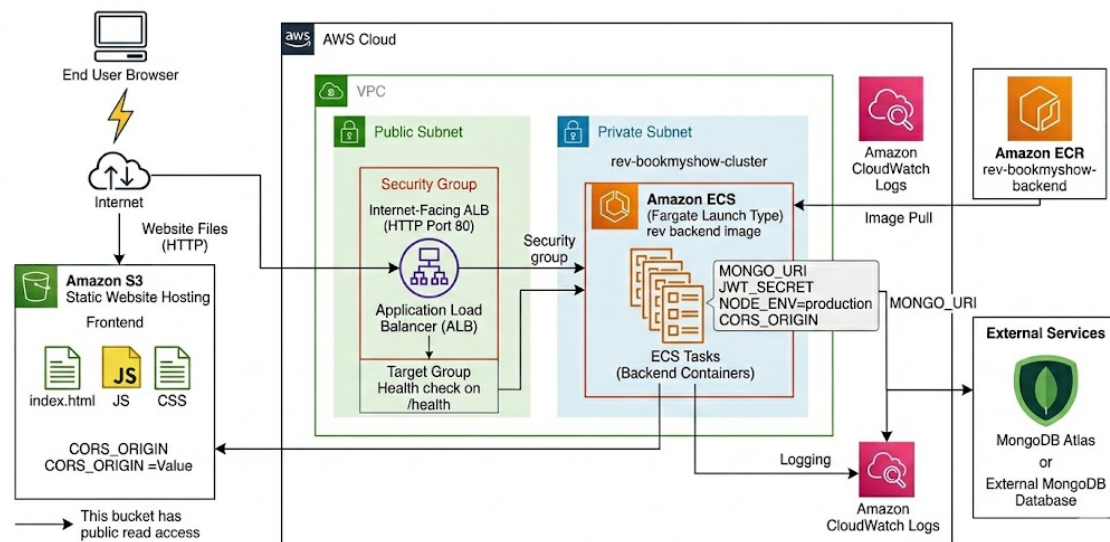


Figure 1: Final AWS deployment architecture overview

2. Prerequisites

Before deployment, the following tools were installed locally:

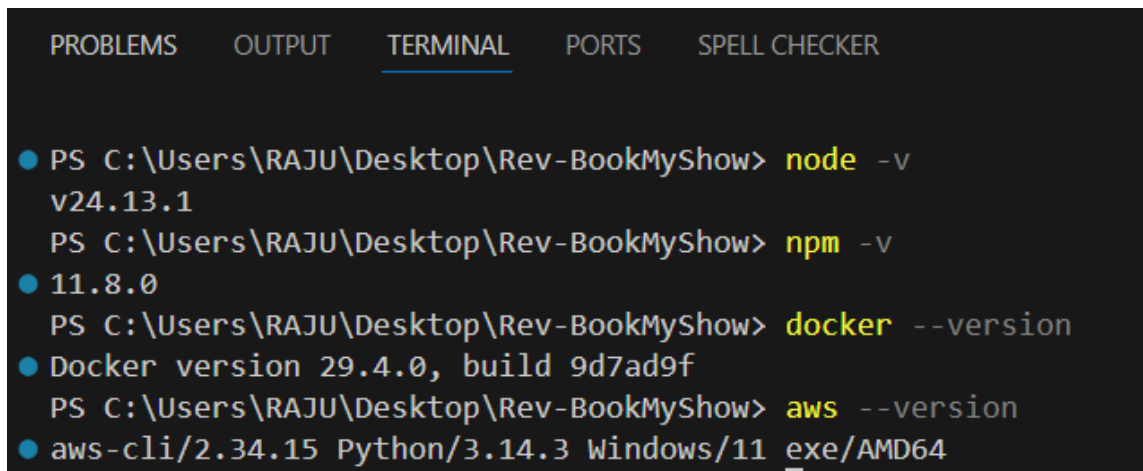
- Node.js 18 or higher
- npm
- Docker Desktop
- AWS CLI
- Git

The following AWS permissions were required:

- Amazon ECR access
- Amazon ECS access
- Load balancer and security group configuration
- S3 bucket management
- CloudWatch logs access

2.1. Tool Verification

The local environment was checked before deployment.



```
PROBLEMS  OUTPUT  TERMINAL  PORTS  SPELL CHECKER

● PS C:\Users\RAJU\Desktop\Rev-BookMyShow> node -v
v24.13.1
PS C:\Users\RAJU\Desktop\Rev-BookMyShow> npm -v
● 11.8.0
PS C:\Users\RAJU\Desktop\Rev-BookMyShow> docker --version
● Docker version 29.4.0, build 9d7ad9f
PS C:\Users\RAJU\Desktop\Rev-BookMyShow> aws --version
● aws-cli/2.34.15 Python/3.14.3 Windows/11 exe/AMD64
```

Figure 2: Local tool versions verified in terminal

```

● aws-cli/2.34.15 Python/3.14.3 Windows/11 exe/AMD64
PS C:\Users\RAJU\Desktop\Rev-BookMyShow> aws configure
● AWS Access Key ID [*****WAVP]:
AWS Secret Access Key [*****U2LZ]:
Default region name [us-east-1]:
Default output format [None]:
PS C:\Users\RAJU\Desktop\Rev-BookMyShow> aws sts get-caller-identity
● {
  "UserId": "AIDAZ2U5LMDU3GRQJEWQO",
  "Account": "675713605865",
  "Arn": "arn:aws:iam::675713605865:user/chintu"
}

```

Figure 3: AWS CLI identity confirmation

3. Backend Deployment

The backend was containerized and deployed using ECR, ECS Fargate, and an Application Load Balancer.

3.1. Docker Image Build

The backend Docker image was built from the backend folder.

```

PS C:\Users\RAJU\Desktop\Rev-BookMyShow\backend> docker build -t rev-bookmyshow-backend .
=> => extracting sha256:72af4232fa8b9b6c46a4465e10721e29fcd874865e23f2d6ec0d73b073fb145a
=> => extracting sha256:901b3601b2ce3a920055d1d6c599139d03ba507fe4d0b07c86976d028c311c34
=> => extracting sha256:d49944ef3e70b10c61b930eda7d0338965838edece41d5733d68bc123ac9ebc6
=> [internal] load build context
=> => transferring context: 1.37kB
=> [runtime 2/3] WORKDIR /app
=> [build 2/5] WORKDIR /app
=> [build 3/5] COPY package*.json ./
=> [build 4/5] RUN npm install --omit=dev --no-package-lock
=> [build 5/5] COPY . .
=> [runtime 3/3] COPY --from=build /app /app
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:8c3b48e0a154ecd3188bfc53001129a67df717c3045740b4d9de3333e2cc3cac
=> => exporting config sha256:d29998df46e4adb1408fa551c867eb86664974be2cacadccaf36060f3a4217de
=> => exporting attestation manifest sha256:75953f5c19455f0e5473084550006fb49ac4062c7377d7073ddbec3c645a7f72
=> => exporting manifest list sha256:54d9aed14a2e9a8138ac720d7c13f2a53e30c49564890f04a95209d1994b09ec
=> => naming to docker.io/library/rev-bookmyshow-backend:latest
=> => unpacking to docker.io/library/rev-bookmyshow-backend:latest

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/jzdaftlirsg7wx5eu04tmr

```

Figure 4: Successful backend Docker image build

3.2. ECR Repository

An Amazon ECR repository named `rev-bookmyshow-backend` was created to store the backend image.

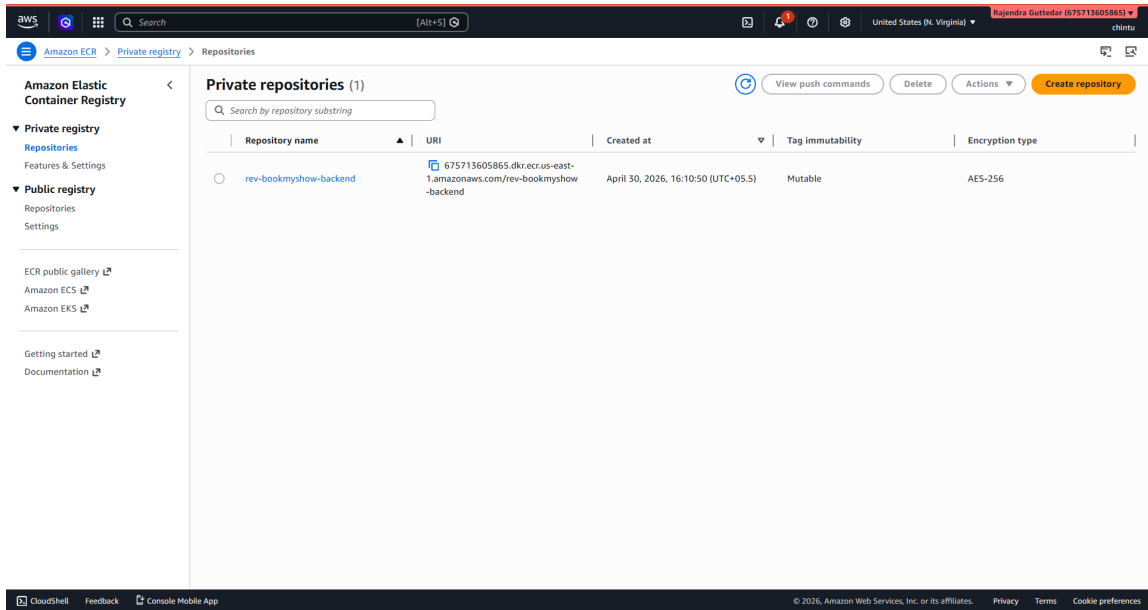


Figure 5: ECR repository created for backend image storage

3.3. Image Push to ECR

After building, the image was tagged and pushed to ECR.

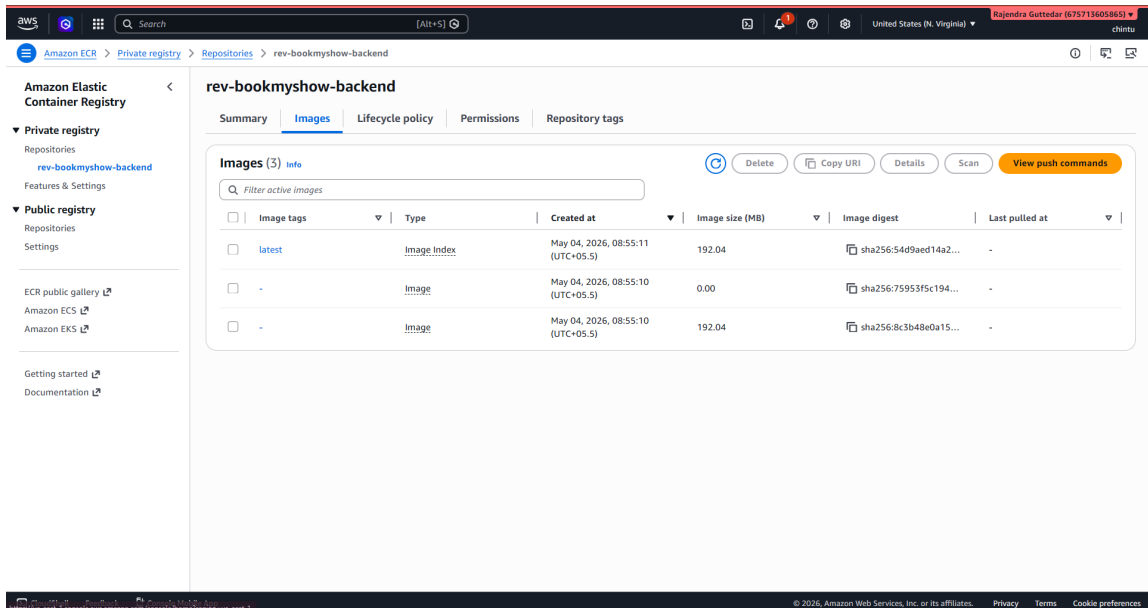


Figure 6: Backend image pushed to ECR with latest tag

3.4. ECS Cluster

An ECS Fargate cluster named `rev-bookmyshow-cluster` was created for running the backend container.

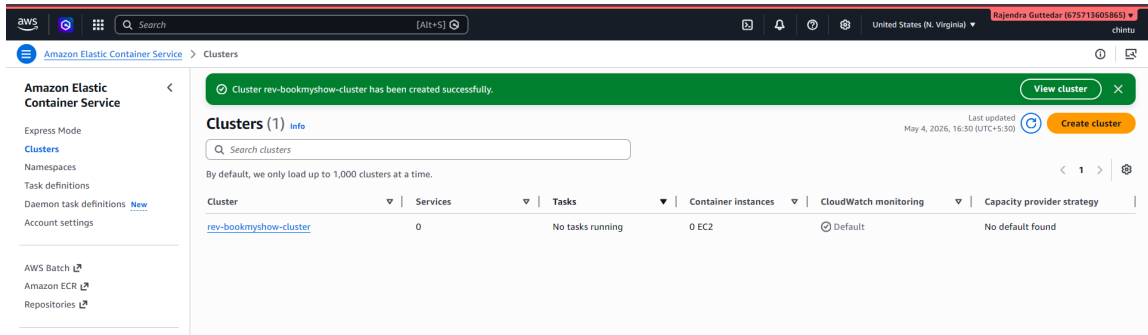


Figure 7: ECS cluster created successfully

3.5. Task Definition

The ECS task definition included the backend container, port mapping, and environment variables.

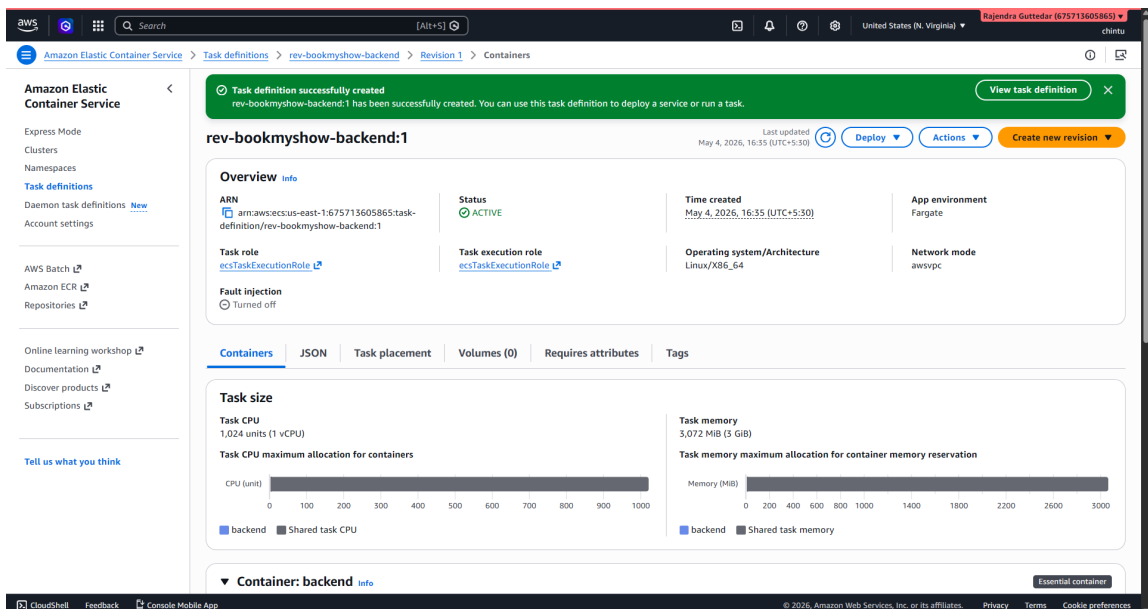


Figure 8: Task definition container settings

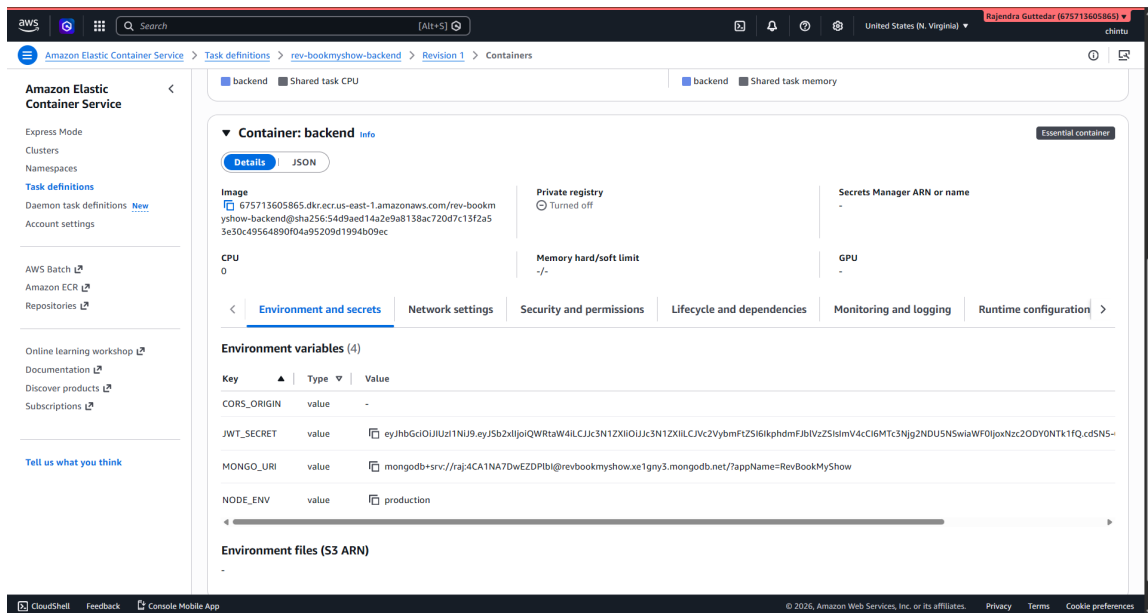


Figure 9: Task definition environment variables

Important environment variables used in production:

- MONGO_URI
- JWT_SECRET
- NODE_ENV=production
- CORS_ORIGIN=http://<your-s3-website-endpoint>

3.6. Application Load Balancer

The backend service was exposed through an internet-facing Application Load Balancer listening on port 80.

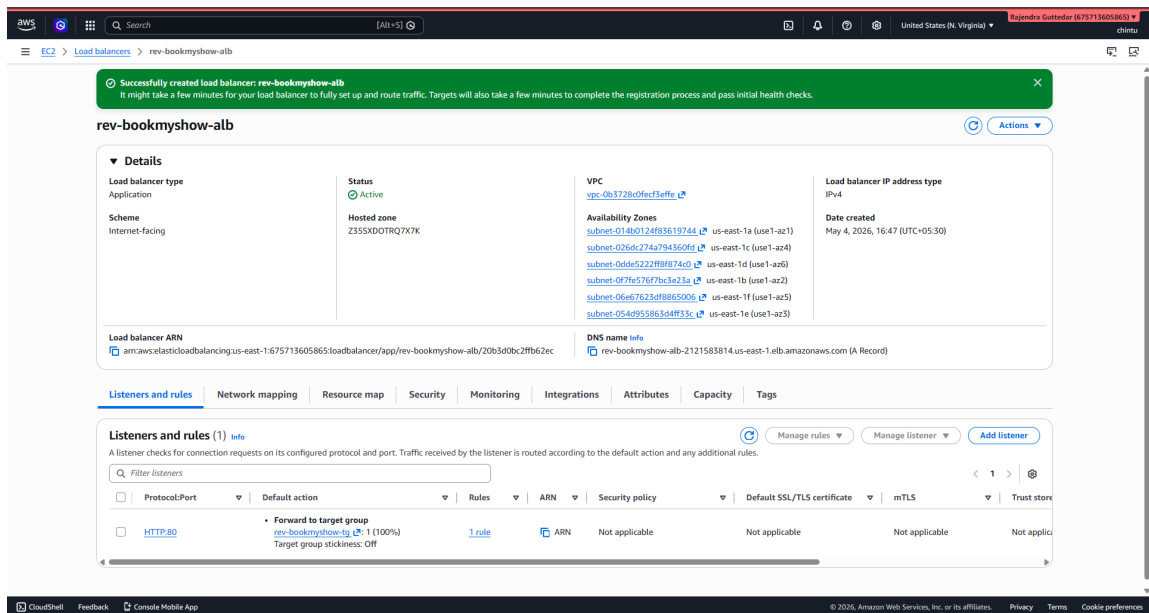


Figure 10: Application Load Balancer listener configuration

3.7. Target Group

The target group health check was configured to use the backend health endpoint.

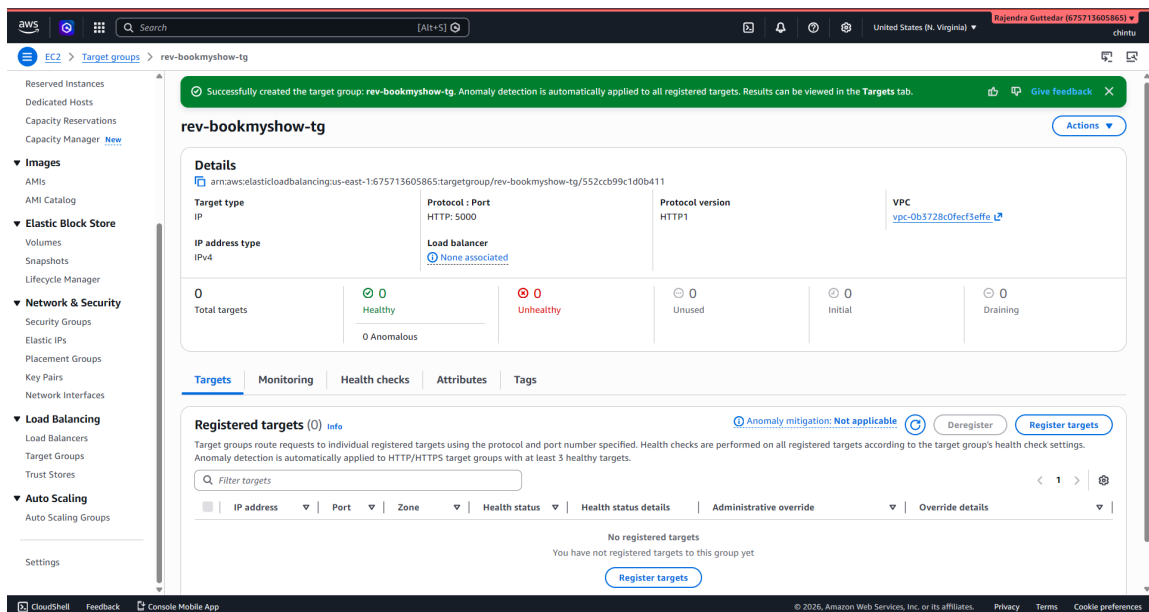


Figure 11: Target group health check configuration

3.8. ECS Service

The ECS service was created using the task definition and attached to the load balancer target group.

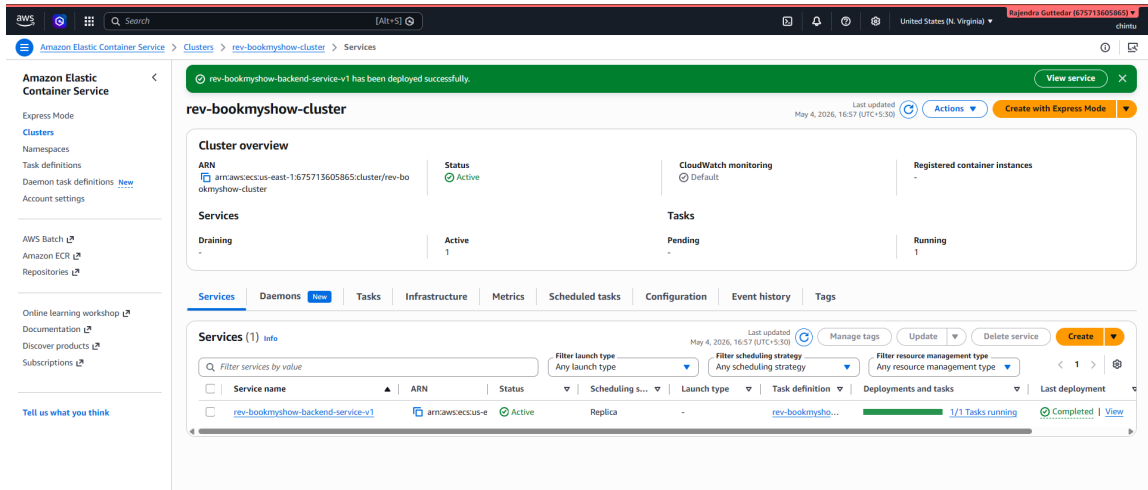


Figure 12: ECS service running successfully

3.9. Backend Health Verification

The backend health endpoint was tested through the load balancer.

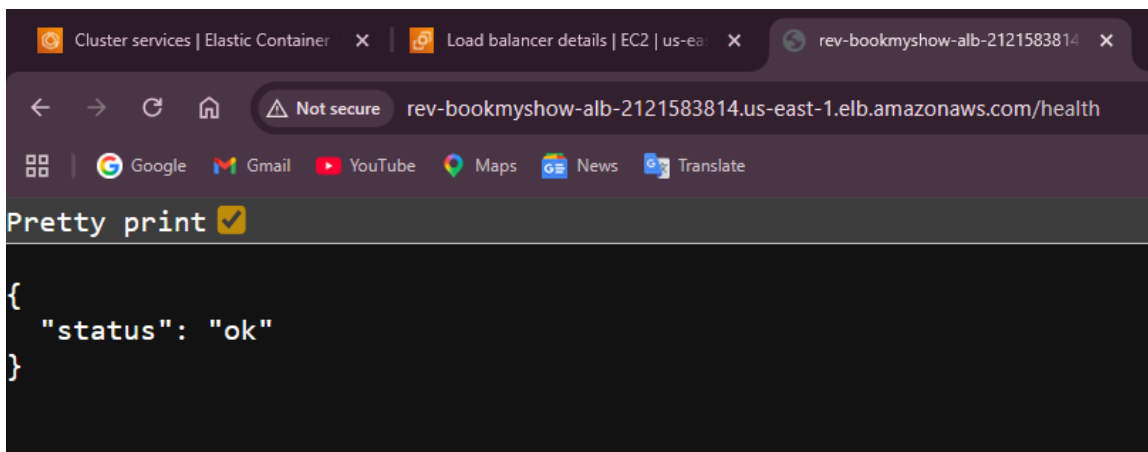


Figure 13: Successful backend health response through ALB

4. Frontend Deployment

The frontend was built using Vite and deployed as a static website on Amazon S3.

4.1. Frontend API Configuration

The frontend environment file was updated to point to the backend ALB endpoint.

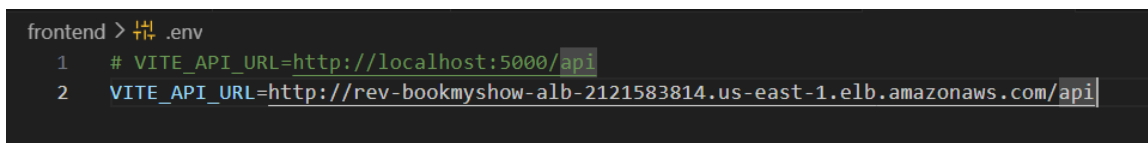
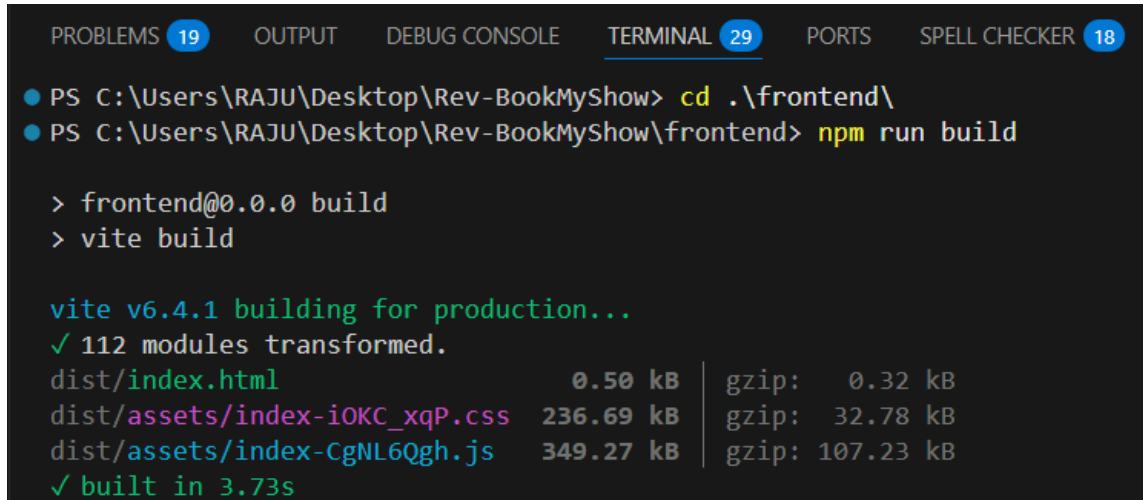


Figure 14: Frontend environment file configured with backend API URL

4.2. Frontend Build

The frontend was built before uploading the static files to S3.



```
PROBLEMS 19 OUTPUT DEBUG CONSOLE TERMINAL 29 PORTS SPELL CHECKER 18

● PS C:\Users\RAJU\Desktop\Rev-BookMyShow> cd .\frontend\
● PS C:\Users\RAJU\Desktop\Rev-BookMyShow\frontend> npm run build

> frontend@0.0.0 build
> vite build

vite v6.4.1 building for production...
✓ 112 modules transformed.
dist/index.html 0.50 kB | gzip: 0.32 kB
dist/assets/index-iOKC_xqP.css 236.69 kB | gzip: 32.78 kB
dist/assets/index-CgNL6Qgh.js 349.27 kB | gzip: 107.23 kB
✓ built in 3.73s
```

Figure 15: Frontend build completed successfully

4.3. S3 Bucket Creation

An S3 bucket was created for static website hosting.

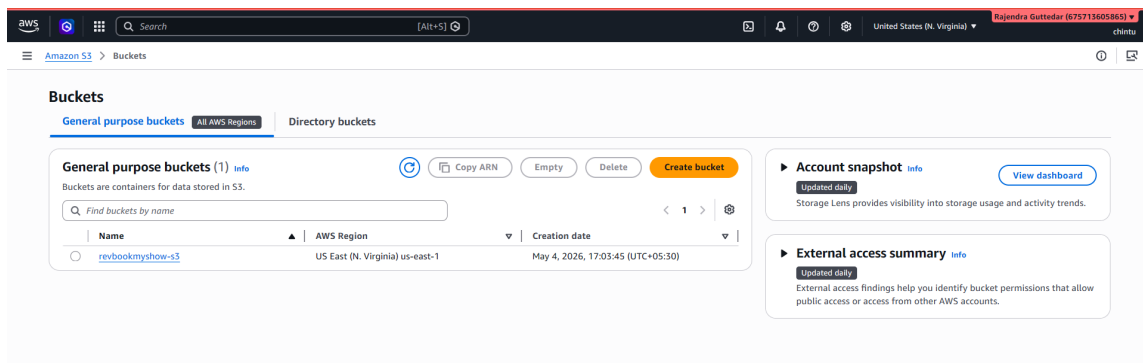


Figure 16: S3 bucket created for frontend hosting

4.4. Static Website Hosting

Static website hosting was enabled and the index and error documents were set to `index.html`.

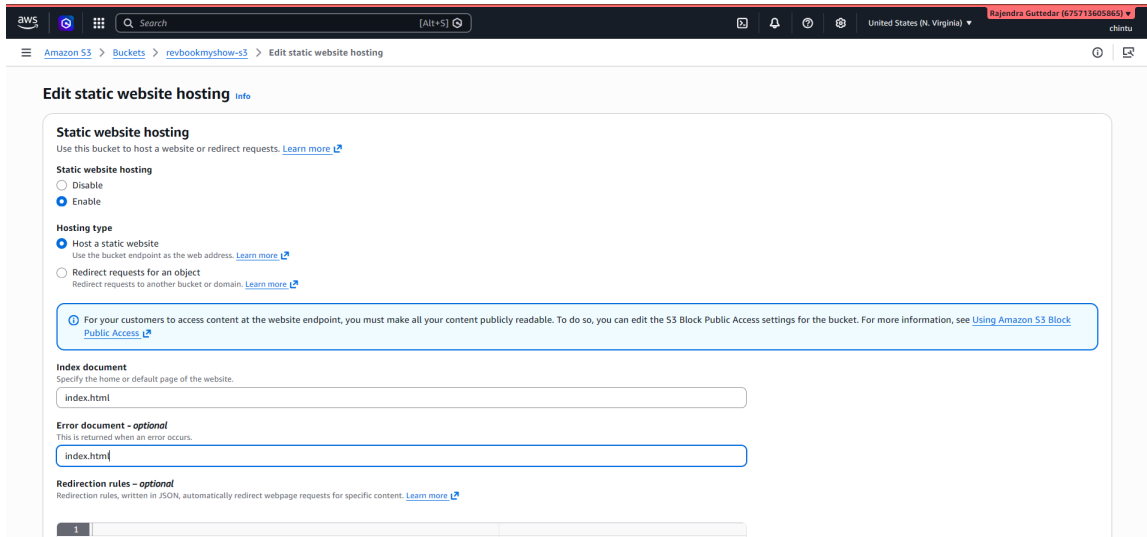


Figure 17: S3 static website hosting configuration

4.5. Public Access and Bucket Policy

To allow public website access, block public access was disabled and a public read bucket policy was added.

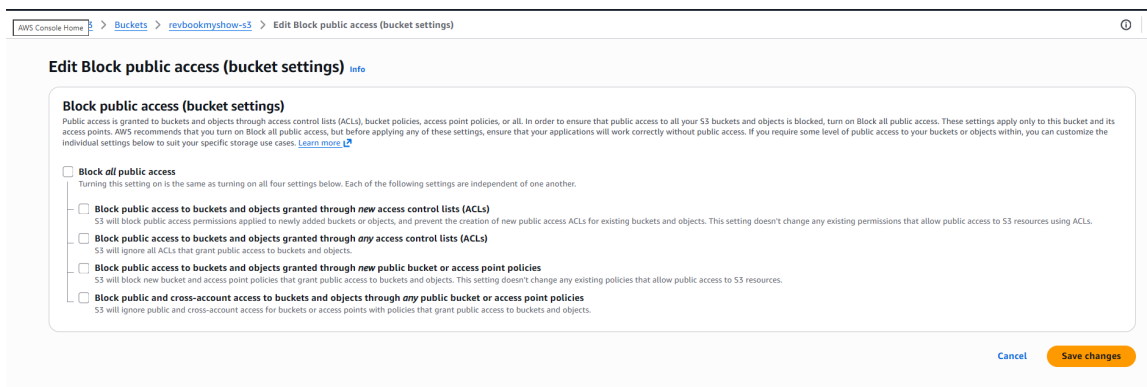


Figure 18: Block public access disabled for website hosting

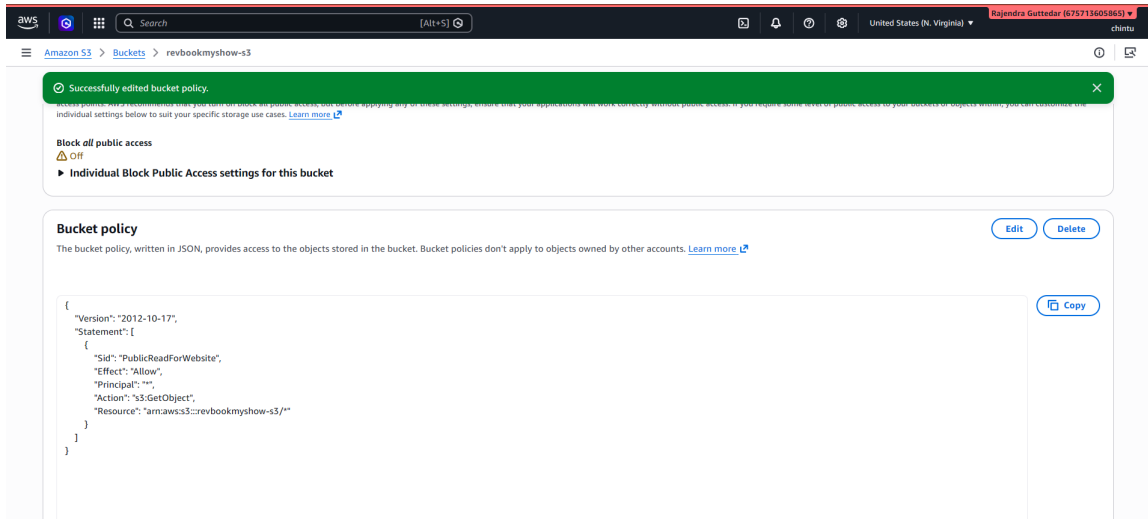


Figure 19: Bucket policy saved to allow public read access

4.6. Uploading Dist Files

The contents of the frontend `dist` folder were uploaded directly to the S3 bucket root.

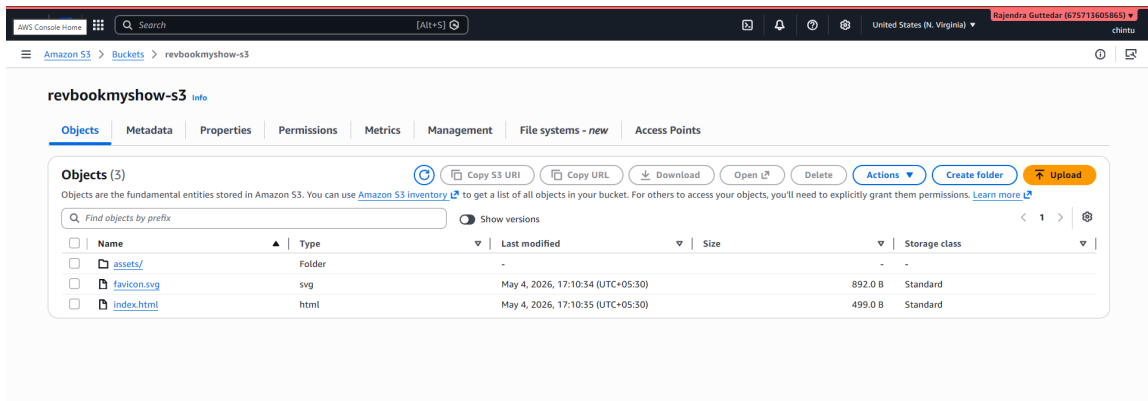


Figure 20: Uploaded frontend files with index.html at bucket root

4.7. Frontend Live URL

The deployed frontend was opened using the S3 website endpoint.

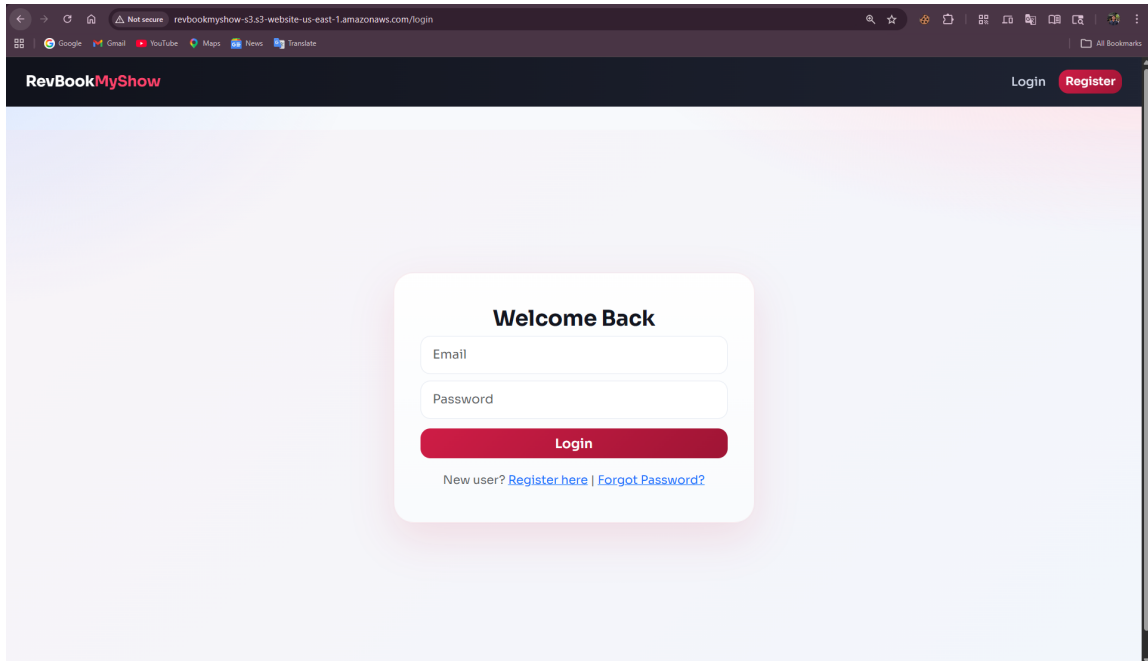


Figure 21: Frontend loaded successfully from the S3 website URL

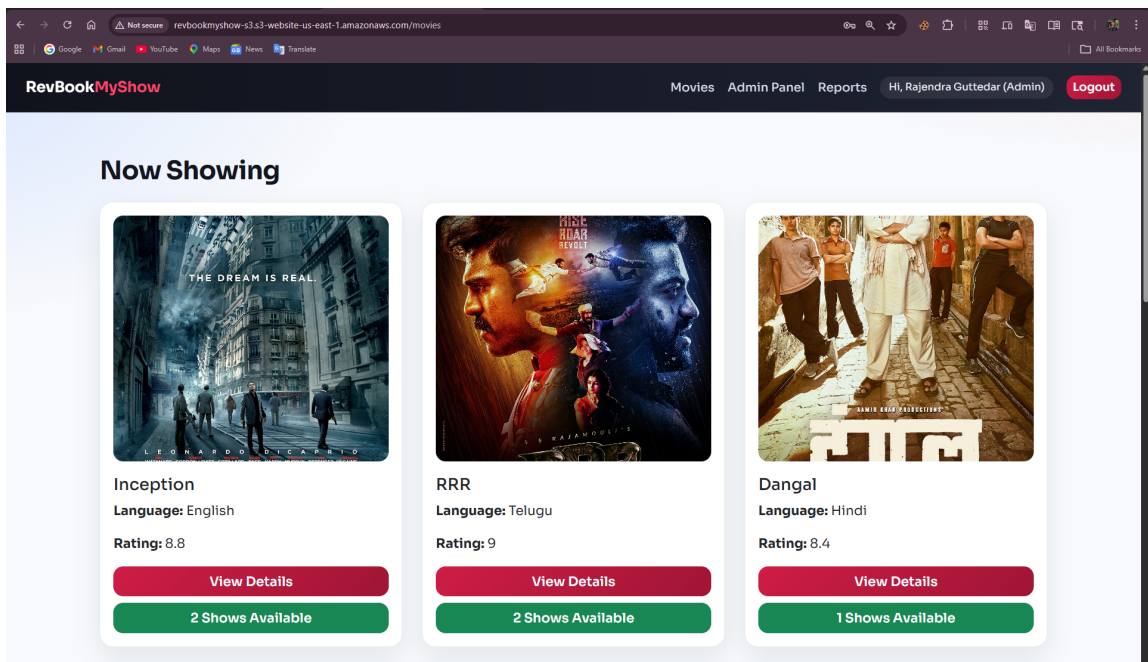


Figure 22: Alternative screenshot of frontend live website URL

5. CORS and Environment Matching

The backend `CORS_ORIGIN` value was matched exactly with the S3 website origin. This step is important because login and registration fail if the frontend origin does not match the backend CORS configuration.

- `VITE_API_URL` points to the ALB `/api` endpoint
- The frontend was rebuilt and reuploaded after the change

6.4. Mixed Content Error

The final deployment intentionally uses HTTP S3 website hosting and HTTP ALB backend to avoid mixed content issues between HTTPS and HTTP resources.

7. Conclusion

This deployment successfully hosted the RevBookMyShow frontend on Amazon S3 and the backend on Amazon ECS Fargate behind an Application Load Balancer, with container images stored in Amazon ECR. The architecture is simple, reproducible, and suitable for project demonstration.